

Site e-commerce Symfony : Esprit Déco

Lot 3 : Front Office – Passer une commande

- Constitution et gestion d'un panier (affichage, ajout, suppression, modification des quantités)
- Tunnel de vente pour passer commande
- Paiement en ligne

Comprendre la gestion d'une commande

Pour gérer une commande sur un site e-commerce, on a besoin des entités User, Product et de deux nouvelles entités :

- Order (qui représente la commande)
- OrderLine (qui représente les lignes de la commande)

Un utilisateur (**User**) peut passer 0 ou plusieurs commande (**Order**) et 1 commande appartient à 1 et 1 seul utilisateur : **User (0,N)-- passe – (1,1) Order**

Une commande (**Order**) peut contenir 1 ou plusieurs lignes (**OrderLine**) et une ligne appartient à 1 et 1 seule commande : **Order (1,N)--contient--(1,1) OrderLine**

Une ligne (**OrderLine**) concerne 1 et 1 seul produit (**Product**) et 1 produit appartient à 0 ou plusieurs lignes : **OrderLine (1,1) –concerne--(0,N) Product**

Order : id , user, createdAt (DateTimeImmutable), status (« pending_payment », « paid », « shipped », « delivered », « canceled »), total_amount, orderLines

OrderLine : id, order, product, quantity (integer), unitPrice (float)

OrderLine est un élément clé dans la gestion d'une commande. Elle permet de stocker le produit ajouté au panier et sa quantité.

Exemple de commande :

- 3 tomates
- 4 carottes
- 1 chou

Dans la BDD, 3 lignes seront enregistrées dans la table Order_line :

id	Order_id	product_id	quantity	unit_price
1	13	7 (id de la tomate)	3	3
2	13	4 (id de la carotte)	4	2
3	13	5 (id du chou)	1	4,5

« Price » représente le prix unitaire du produit au moment de la validation de la commande.

Comprendre la gestion du panier

Avant de passer une commande, l'utilisateur constitue son panier. Ce n'est qu'au moment de la validation de son panier que la commande sera créée.

Pour gérer ce panier, on a besoin des entités User, Product et de deux nouvelles entités :

- Cart (qui représente la commande) qui ressemble beaucoup à Order
- CartLine (qui représente les lignes du panier) qui ressemble beaucoup à OrderLine

Un utilisateur (**User**) peut constituer 0 ou 1 seul panier (**Cart**) et 1 panier appartient à 1 et 1 seul utilisateur : **User (0,1)-- constitue –(1,1) Cart**

Un panier (**Cart**) peut contenir 1 ou plusieurs lignes (**CartLine**) et une ligne appartient à 1 et 1 seul panier : **Cart (1,N)-- contient--(1,1) CartLine**

Une ligne (**CartLine**) concerne 1 et 1 seul produit (**Product**) et 1 produit appartient à 0 ou plusieurs lignes : **CartLine (1,1) –concerne--(0,N) Product**

Cart : id , user, status (« open », « converted »), cartLines

CartLine : id, cart, product, quantity (integer)

En pratique, pour constituer son panier, lorsque l'utilisateur y ajoute un produit :

- un nouveau panier est créé (s'il n'existe pas déjà)
- une nouvelle CartLine est ajoutée dans ce panier auquel elle est liée.

1) Constitution du panier

a) Ajouter un produit au panier

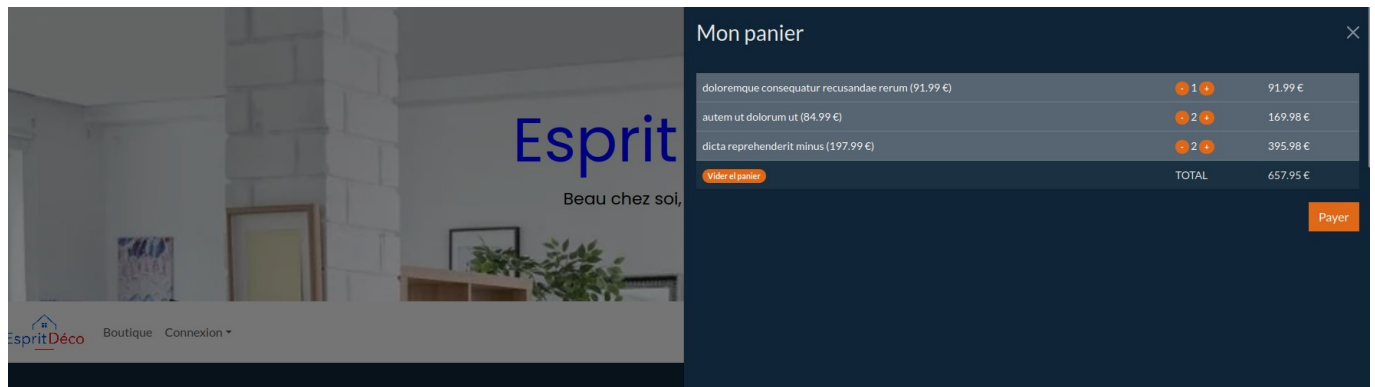
Ajouter à droite de la barre de navigation un panier avec un indicateur du nombre d'articles.



Dans la fiche d'un produit, si on clique sur « ajouter au panier » :

- l'indicateur du nombre d'articles s'incrémente
- le panier est sauvegardé en session

Lorsqu'on clique sur le panier, un panneau latéral (offcanvas) apparaît à droite de l'écran et récapitule la commande.



Aide à la réalisation :

- Modifier le bouton « ajouter au panier » et créer le CartController avec la méthode add() pour enregistrer en session dans « cart » un produit ajouté au panier.
- « cart » est un tableau qui représente le panier et stocke en clé l'id du produit ajouté et en valeur la quantité de ce produit.

exemple de panier:

[

```
8 => 1      // produit d'id 8 => 1 unité
29 => 2     // produit d'id 29 => 2 unité
13 => 4     // produit d'id 13 => 4 unité
```

]

- Gérer les sessions : <https://symfony.com/doc/current/session.html>
 - Pour récupérer le panier « cart » depuis la session (renvoie [] si « cart » n'existe pas) :
`$cart = $session->get('cart', []);`
 - Pour modifier « cart » dans la session : `$session->set('cart', $cart);`
 - Pour suivre l'évolution de votre panier, placer un dump de « cart » dans le contrôleur qui affiche tous les produits : `dump($session->get('cart'));`
 - Pour vider le panier : `$session->remove('cart');`
- Créer la vue offcanvas et modifier la vue barre de navigation
- Afficher le produit dans le panier
Pour cela on va créer une méthode getCart() dans un CartHandler qu'on pourra directement appeler dans notre offcanvas avec Twig (voir doc symfony)
<https://symfony.com/doc/current/templates.html#global-variables>

⚠ On ne crée pas d'entité Cart ici. On le fera dans le 1)c lorsqu'on aura besoin d'enregistrer en BDD.

b) Gestion du panier

Dans le offcanvas, activer les bouton -, +, 🗑️ et vider le panier.

👉👉👉 La bonne pratique est que ces boutons appellent via AJAX les méthodes de CartController (add, decrease, remove et clear)

c) 👉👉👉 Enregistrement en BDD du panier pour un utilisateur connecté

Afin que le panier de l'utilisateur soit conservé même après une déconnexion, on souhaite que le panier soit enregistré en BDD.

En pratique, si l'utilisateur n'est pas connecté, le panier est stocké **uniquement** en session. Alors que s'il est connecté le panier n'est pas stocké en session, mais directement et **uniquement** en BDD.

⚠️ Attention, on a qu'une seule source de vérité.

- la session, si l'utilisateur n'est pas connecté
- la bdd, si l'utilisateur est connecté

C'est l'un ou l'autre, mais jamais les deux en même temps.

Aide à la réalisation :

Créer les entités Cart et CartLine.

Dans les méthodes du CartController, il faut vérifier si l'utilisateur est connecté pour enregistrer en session ou en bdd.

Dans le cas où l'utilisateur se connecte après avoir commencé à constituer un panier, c'est au moment de la connexion que le panier en session sera enregistré en BDD.

Pour faire cela, on va utiliser un EventSubscriber qui va écouter la connexion. En cas de connexion, il va enregistrer en BDD un panier (s'il existe déjà en session) et le supprimer de la session.

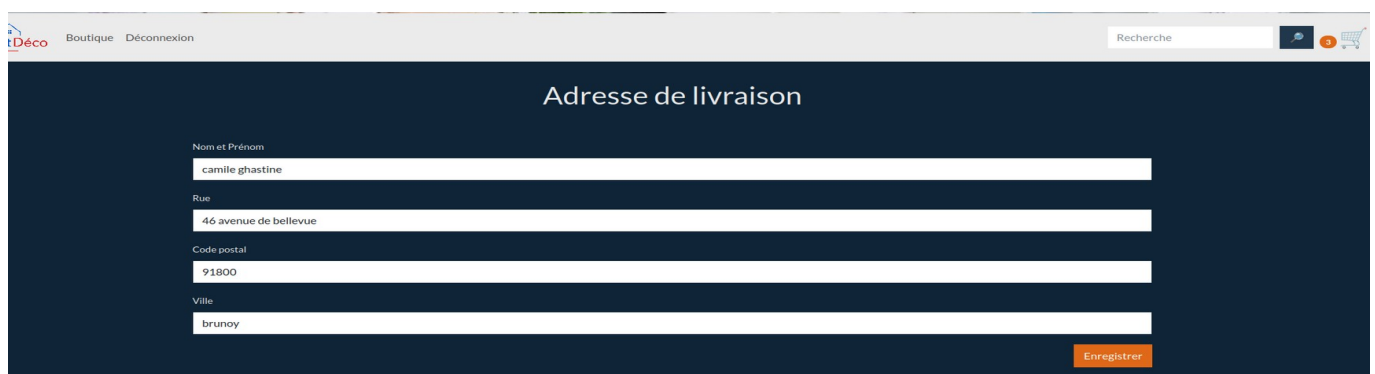
https://symfony.com/doc/current/event_dispatcher.html#creating-an-event-subscriber

2) Tunnel de vente

a) Adresse de livraison

Dans le panier, lorsque l'utilisateur clique sur « payer », il est redirigé sur une page pour renseigner son adresse de livraison.

Entité **Address** : id, fullname, street, zipCode, City, order (relation OneToOne)



Aide à la réalisation :

Lors de l'enregistrement de l'adresse, un « order » est créé et persisté en BDD.

Le user, le createdAt et le status de l'ordre sont définis.

b) Récapitulatif de la commande

Lorsque l'utilisateur enregistre son adresse, il doit valider sa commande.

The screenshot displays a web interface for order validation. At the top, there's a navigation bar with 'Déco', 'Boutique', 'Déconnexion', and a search bar labeled 'Recherche'. The main heading is 'Validation de la commande'. Below it, the section 'Récapitulatif de votre comande' contains a table with three columns: item name, quantity and unit price, and total price.

maxime	1 x 170.99 €	170.99 €
dolorem veritatis	1 x 87.99 €	87.99 €
consequuntur saepe est explicabo illum	1 x 24.99 €	24.99 €
TOTAL		283.97 €

Below the table is the 'Adresse de livraison' section, which includes a form with the following text: 'toto', '3 rue de la gare', '75001', and 'Paris'. At the bottom of this section are two buttons: 'Modifier l'adresse de livraison' and 'Confirmer'.

c) Paiement

Une fois la commande confirmée, l'utilisateur est amené vers le terminal de payment Stripe pour payer.